Javascript

AULA 06

PROFESSOR MAROMO

Javascript

DOM

Nesta aula veremos o conceito de Document Object Model

Agenda

- DOM
- Seleção e manipulação de elementos
- Eventos

DOM. O que é?

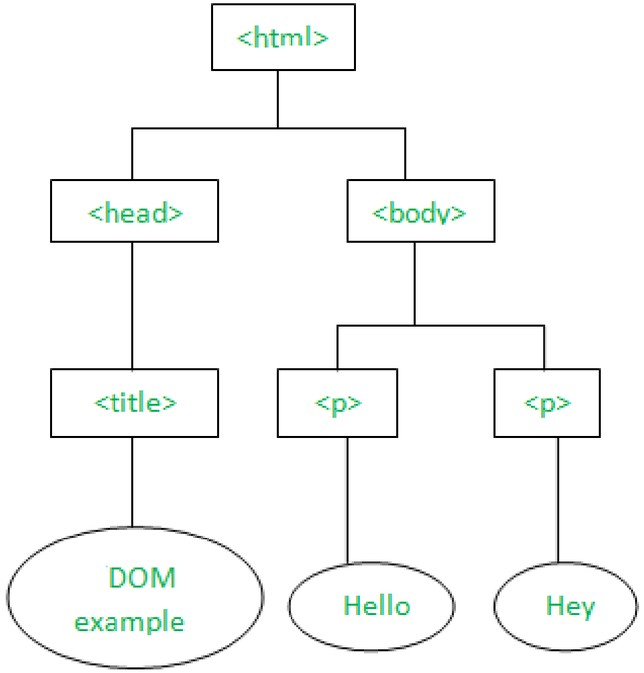
O DOM (Document Object Model) é uma interface de programação que representa a estrutura de um documento HTML/XML em forma de uma árvore de objetos. O DOM permite que você acesse e manipule os elementos, atributos e conteúdos do documento por meio de scripts em JavaScript.

DOM

Basicamente, o DOM transforma o documento HTML/XML em uma estrutura de objetos, onde cada elemento, atributo e texto é representado por um nó na árvore de objetos. Essa representação em forma de árvore permite que você navegue pelos elementos, acesse suas propriedades, altere seu conteúdo e estilize-os dinamicamente.

DOM

```
<html>
<head>
<title>
 DOM example
</title>
</head>
<body>
<p id = "para1">Hello</p>
<p id =  “para2”>Hey</p>
</body>
</html>
```



Escopo de Funções

Antes de prosseguir com os detalhes sobre o escopo de funções em JavaScript, é importante entender a importância desse conceito no desenvolvimento de programas. O escopo desempenha um papel fundamental na organização e no controle de acesso às variáveis em um programa.

Seleção e manipulação de elementos

Para selecionar elementos no DOM, você pode usar métodos e propriedades disponíveis no JavaScript.

Alguns métodos

Método	Descrição
getElementById(id)	Retorna o elemento com o ID especificado.
getElementsByClassName(className)	Retorna uma coleção de elementos com a classe especificada.
getElementsByTagName(tagName)	Retorna uma coleção de elementos com a tag especificada.
querySelector(selector)	Retorna o primeiro elemento que corresponde ao seletor CSS especificado.
querySelectorAll(selector)	Retorna uma lista de todos os elementos que correspondem ao seletor CSS especificado.

Manipulando eventos

Uma vez selecionado o elemento desejado, você pode manipulá-lo de várias maneiras, como:

- Acessar e modificar propriedades, como innerHTML, textContent, value, src, href, entre outras.
- Adicionar ou remover classes com as propriedades classList e className.
- Modificar estilos utilizando a propriedade style.
- Adicionar ou remover atributos utilizando as propriedades setAttribute e removeAttribute.
- Manipular eventos, como clique, submissão de formulário, mouseover, etc.

Eventos

Os eventos permitem que você responda a ações do usuário ou ações que ocorrem no documento. Por exemplo, quando o usuário clica em um botão, um evento de clique é acionado, permitindo que você execute uma função em resposta a esse evento.

Eventos

No JavaScript, você pode registrar um ouvinte de eventos (event listener) em um elemento específico para detectar e responder a um evento específico. Existem várias maneiras de adicionar ouvintes de evento. Veja como:

Ex:

- Utilizando a propriedade on<evento> diretamente no elemento HTML.
- Usando o método addEventListener para adicionar ouvintes de eventos de forma mais flexível.

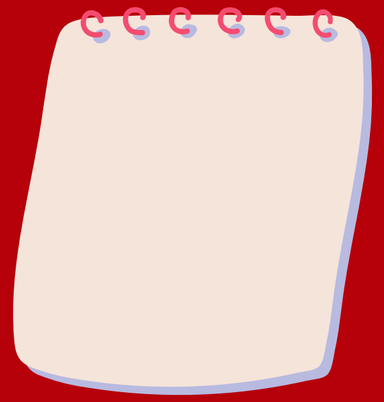
Evento

```javascript
let button = document.querySelector('button');

button.addEventListener('click', function() {
  console.log('O botão foi clicado!');
});
```

Nesse exemplo, um ouvinte de evento de clique é adicionado ao botão selecionado. Quando o botão for clicado, a função passada como argumento será executada, exibindo uma mensagem no console.

Nota:



Apresenta-se, nos próximos slides, 10 exemplos de uso dos métodos de seleção do DOM, acompanhados de trechos de código HTML para ilustrar cada caso. Estão descritos o que se pretende fazer e, em seguida, é apresentado o documento HTML correspondente.

Eventos e Manipulação do DOM

Exemplo 1

Exemplo usando getElementById(id): Pretendemos selecionar um elemento de uma página pelo seu ID.

html Copy code

```html
<!DOCTYPE html>
<html>
  <head>
    <title>Exemplo getElementById</title>
  </head>
  <body>
    <h1 id="titulo">Título</h1>
    <script>
      let elemento = document.getElementById("titulo");
      console.log(elemento.innerText); // Exibe o texto "Título" no consc
    </script>
  </body>
</html>
```

Exemplo 2

Exemplo usando getElementsByClassName(className):
Pretendemos selecionar todos os elementos de uma página que possuam uma determinada classe

```html
<!DOCTYPE html>
<html>
  <head>
    <title>Exemplo getElementsByClassName</title>
  </head>
  <body>
    <p class="paragrafo">Parágrafo 1</p>
    <p class="paragrafo">Parágrafo 2</p>
    <script>
      let elementos = document.getElementsByClassName("paragrafo");
      for (let i = 0; i < elementos.length; i++) {
        console.log(elementos[i].innerText);
      }
      // Exibe "Parágrafo 1" e "Parágrafo 2" no console
    </script>
  </body>
</html>
```

Exemplo 3

Exemplo usando getElementsByTagName(tagName):
Pretendemos selecionar todos os elementos de uma página que tenham uma determinada tag.

```html
<!DOCTYPE html>
<html>
  <head>
    <title>Exemplo getElementsByTagName</title>
  </head>
  <body>
    <h2>Subtítulo</h2>
    <p>Parágrafo 1</p>
    <p>Parágrafo 2</p>
    <script>
      let elementos = document.getElementsByTagName("p");
      for (let i = 0; i < elementos.length; i++) {
        console.log(elementos[i].innerText);
      }
      // Exibe "Parágrafo 1" e "Parágrafo 2" no console
    </script>
  </body>
</html>
```

Exemplo 4

Exemplo usando querySelector(selector): Pretendemos selecionar o primeiro elemento de uma página que corresponda a um seletor CSS específico.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Exemplo querySelector</title>
  </head>
  <body>
    <div class="container">
      <h3>Título</h3>
      <p>Conteúdo do parágrafo</p>
    </div>
    <script>
      let elemento = document.querySelector(".container p");
      console.log(elemento.innerText); // Exibe "Conteúdo do parágrafo" r
    </script>
  </body>
</html>
```

Exemplo 5

Exemplo usando querySelectorAll(selector): Pretendemos selecionar todos os elementos de uma página que correspondam a um seletor CSS específico.

```html
<!DOCTYPE html>
<html>
  <head>
    <title>Exemplo querySelectorAll</title>
  </head>
  <body>
    <ul>
      <li>Item 1</li>
      <li>Item 2</li>
      <li>Item 3</li>
    </ul>
    <script>
      let elementos = document.querySelectorAll("li");
      for (let i = 0; i < elementos.length; i++) {
        console.log(elementos[i].innerText);
      }
      // Exibe "Item 1", "Item 2" e "Item 3" no console
    </script>
  </body>
</html>
```

Exemplo 6

Exemplo usando getElementById(id) com manipulação de conteúdo: Pretendemos selecionar um elemento pelo seu ID e alterar seu conteúdo.

```html
<!DOCTYPE html>
<html>
  <head>
    <title>Exemplo getElementById com manipulação de conteúdo</title>
  </head>
  <body>
    <h1 id="titulo">Título</h1>
    <script>
      let elemento = document.getElementById("titulo");
      elemento.innerText = "Novo Título";
    </script>
  </body>
</html>
```

Exemplo 7

Exemplo usando getElementsByClassName(className) com manipulação de estilo: Pretendemos selecionar todos os elementos de uma classe e alterar seu estilo.

```html
<!DOCTYPE html>
<html>
  <head>
    <title>Exemplo getElementsByClassName com manipulação de estilo</title
    <style>
      .paragrafo {
        color: blue;
      }
    </style>
  </head>
  <body>
    <p class="paragrafo">Parágrafo 1</p>
    <p class="paragrafo">Parágrafo 2</p>
    <script>
      let elementos = document.getElementsByClassName("paragrafo");
      for (let i = 0; i < elementos.length; i++) {
        elementos[i].style.fontWeight = "bold";
      }
    </script>
  </body>
</html>
```

Exemplo 8

Exemplo usando getElementsByTagName(tagName) com manipulação de atributos: Pretendemos selecionar todos os elementos de uma tag e alterar um atributo específico.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Exemplo getElementsByTagName com manipulação de atributos</tit
  </head>
  <body>
    <img src="imagem1.jpg">
    <img src="imagem2.jpg">
    <script>
      let elementos = document.getElementsByTagName("img");
      for (let i = 0; i < elementos.length; i++) {
        elementos[i].setAttribute("alt", "Imagem");
      }
    </script>
  </body>
</html>
```

Exemplo 9

Exemplo usando querySelector(selector) com manipulação de classes: Pretendemos selecionar o primeiro elemento correspondente a um seletor CSS e adicionar/remover classes.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Exemplo querySelector com manipulação de classes</title>
    <style>
      .destaque {
        color: red;
      }
    </style>
  </head>
  <body>
    <p>Parágrafo</p>
    <script>
      let elemento = document.querySelector("p");
      elemento.classList.add("destaque"); // Adiciona a classe "destaque
      // elemento.classList.remove("destaque"); // Remove a classe "dest
    </script>
  </body>
</html>
```

Exemplo 10

Exemplo de uso de eventos com addEventListener:
Pretendemos adicionar um ouvinte de evento a um elemento para responder a um evento específico.

```html
<!DOCTYPE html>
<html>
  <head>
    <title>Exemplo de uso de eventos</title>
  </head>
  <body>
    <button id="botao">Clique aqui</button>
    <script>
      let botao = document.getElementById("botao");
      botao.addEventListener("click", function() {
        alert("O botão foi clicado!");
      });
    </script>
  </body>
</html>
```

Conclusão

DOM é essencial para a criação de páginas web interativas, dinâmicas e personalizadas. Ele permite que os desenvolvedores acessem, manipulem e interajam com os elementos de uma página de forma programática, abrindo um leque de possibilidades para a criação de experiências ricas e funcionais. O conhecimento do DOM é fundamental para qualquer desenvolvedor web, pois é a base para o desenvolvimento de aplicações web modernas e responsivas.

FIM

A cada dia, você tem a oportunidade de brilhar e deixar sua marca no mundo